Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 20: OTIMIZANDO TRANSFERÊNCIA DE DADOS ENTRE GPU E CPU

Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 26/09/2026.

1. EXPERIMENTO

Faça o exercício de transferência de calor (heat.c, slide 102) do tutorial de programação de GPUs com OpenMP. Explore as diretivas de movimentação de dados para evitar que esse aspecto domine a execução, evitando a GPU fique muito tempo ociosa e otimizando o desempenho.

2. CONCEITOS RELEVANTES

- **target:** envia uma região de código para execução no dispositivo, neste caso a GPU.
- **teams:** cria equipes de threads para distribuir o trabalho no dispositivo.
- **loop:** indica que as iterações do laço podem ser distribuídas entre os participantes da execução paralela.
- **device(0):** seleciona o dispositivo OpenMP de índice zero.
- **map(to: u[0:n*n]):** mapeia o trecho do array e, ao criar o mapeamento, copia os valores da CPU para a GPU.
- **map(from: u_tmp[0:n*n]):** Mapeia o trecho de saída e, ao encerrar o mapeamento, copia os valores da GPU para a CPU.

- ❖ **map(alloc: u_tmp[0:cells]):** reserva espaço no dispositivo sem transferir os valores iniciais ou finais.

3. MÁQUINA UTILIZADA

No NPAD, para realizar os testes relacionados à tarefa, foi utilizada a partição *gpu-8-v100*.

4. CÓDIGO

Foram desenvolvidas 3 versões, todas usam a mesma fórmula e resolvem o mesmo problema; mudam as transferências e a ordem dos acessos:

- **Original (heatOriginal.c):** envia o campo para a GPU e traz o resultado para a CPU a cada passo. Mantém j no laço interno, acessando a memória com saltos de n posições.
- **Persistente (heatPersistente.c):** mantém os campos na GPU durante todos os passos, com envio inicial e retorno final. Preserva a ordem original dos laços.
- **Contígua (heatContiguo.c)**: mantém os dados na GPU e coloca i no laço interno, acessando posições consecutivas da memória.

Figura 1: heatOriginal.c

```
// Stencil adaptado do código de Tom Deakin (2018); veja heatCommon.c.
#include <stddef.h>
#include "heatCommon.h"

// Atualiza um passo pelo método explícito de diferenças finitas.
// A entrada é somente leitura; cada iteração escreve uma célula da saída.
static void solve(const int n, const double alpha, const double dx, const double dt, const
double * restrict u, double * restrict u_tmp) {

  // Peso dos vizinhos: r; peso da célula central: 1 - 4*r.
  const double r = alpha * dt / (dx * dx);
  const double r2 = 1.0 - 4.0*r;

  // Cada iteração calcula uma célula; os mapas presentes são reutilizados.
```

```
  #pragma omp target teams loop device(0) collapse(2) map(to: u[0:n*n]) map(from:
u_tmp[0:n*n])
  // Ordem original: j interno avança n posições em i+j*n.
  for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
      // Mesmo stencil e mesmas bordas nulas em todas as versões.
      u_tmp[i+j*n] =  r2 * u[i+j*n] +
        r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
        r * ((i > 0)   ? u[i-1+j*n] : 0.0) +
        r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
        r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
    }
  }
}

void evolve(
  int n,
  int nsteps,
  double alpha,
  double dx,
  double dt,
  double **campo,
  double **auxiliar
) {
  double *u = *campo;
  double *u_tmp = *auxiliar;
  for (int t = 0; t < nsteps; ++t) {
    // O kernel termina antes de trocar os campos e iniciar outro passo.
    solve(n, alpha, dx, dt, u, u_tmp);
    double *tmp = u;
    u = u_tmp;
    u_tmp = tmp;
  }
  *campo = u;
  *auxiliar = u_tmp;
}
```

Figura 2: heatPersistente.c

```
// Stencil adaptado do código de Tom Deakin (2018); veja heatCommon.c.
#include <stddef.h>
```

```
#include "heatCommon.h"

// Atualiza um passo pelo método explícito de diferenças finitas.
// A entrada é somente leitura; cada iteração escreve uma célula da saída.
static void solve(const int n, const double alpha, const double dx, const double dt, const
double * restrict u, double * restrict u_tmp) {

  // Peso dos vizinhos: r; peso da célula central: 1 - 4*r.
  const double r = alpha * dt / (dx * dx);
  const double r2 = 1.0 - 4.0*r;

  // Cada iteração calcula uma célula; os mapas presentes são reutilizados.
  #pragma omp target teams loop device(0) collapse(2) map(to: u[0:n*n]) map(from:
u_tmp[0:n*n])
  // Ordem original: j interno avança n posições em i+j*n.
  for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
      // Mesmo stencil e mesmas bordas nulas em todas as versões.
      u_tmp[i+j*n] =  r2 * u[i+j*n] +
        r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
        r * ((i > 0)   ? u[i-1+j*n] : 0.0) +
        r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
        r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
    }
  }
}

void evolve(
  int n,
  int nsteps,
  double alpha,
  double dx,
  double dt,
  double **campo,
  double **auxiliar
) {
  double *u = *campo;
  double *u_tmp = *auxiliar;
  const size_t cells = (size_t)n * n;

  // Envia o estado inicial e mantém as duas alocações na GPU.
```

```
 #pragma omp target data device(0) map(to: u[0:cells]) map(alloc: u_tmp[0:cells])
 {
  for (int t = 0; t < nsteps; ++t) {
    // O kernel termina antes de trocar os campos e iniciar outro passo.
    solve(n, alpha, dx, dt, u, u_tmp);
    double *tmp = u;
    u = u_tmp;
    u_tmp = tmp;
  }
  // Devolve o campo final, para números pares ou ímpares de passos.
  #pragma omp target update device(0) from(u[0:cells])
 }
 *campo = u;
 *auxiliar = u_tmp;
}
```

Figura 3: heatContiguo.c

```
#include <stddef.h>
#include "heatCommon.h"

// Atualiza um passo pelo método explícito de diferenças finitas.
// A entrada é somente leitura; cada iteração escreve uma célula da saída.
static void solve(const int n, const double alpha, const double dx, const double dt, const
double * restrict u, double * restrict u_tmp) {

 // Peso dos vizinhos: r; peso da célula central: 1 - 4*r.
 const double r = alpha * dt / (dx * dx);
 const double r2 = 1.0 - 4.0*r;

 // Cada iteração calcula uma célula; os mapas presentes são reutilizados.
 #pragma omp target teams loop device(0) collapse(2) map(to: u[0:n*n]) map(from:
u_tmp[0:n*n])
 // i interno percorre posições consecutivas em i+j*n.
 for (int j = 0; j < n; ++j) {
  for (int i = 0; i < n; ++i) {
    // Mesmo stencil e mesmas bordas nulas em todas as versões.
    u_tmp[i+j*n] =  r2 * u[i+j*n] +
      r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
      r * ((i > 0)   ? u[i-1+j*n] : 0.0) +
      r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
```

```
      r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
    }
  }
}

void evolve(
  int n,
  int nsteps,
  double alpha,
  double dx,
  double dt,
  double **campo,
  double **auxiliar
) {
  double *u = *campo;
  double *u_tmp = *auxiliar;
  const size_t cells = (size_t)n * n;

  // Envia o estado inicial e mantém as duas alocações na GPU.
  #pragma omp target data device(0) map(to: u[0:cells]) map(alloc: u_tmp[0:cells])
  {
    for (int t = 0; t < nsteps; ++t) {
      // O kernel termina antes de trocar os campos e iniciar outro passo.
      solve(n, alpha, dx, dt, u, u_tmp);
      double *tmp = u;
      u = u_tmp;
      u_tmp = tmp;
    }
    // Devolve o campo final, para números pares ou ímpares de passos.
    #pragma omp target update device(0) from(u[0:cells])
  }
  *campo = u;
  *auxiliar = u_tmp;
}
```

5. RESULTADOS

Abaixo, é possível visualizar o resultado de desempenho do experimento.

Figura 4: Tempo acumulado dos kernels e das transferências entre CPU e GPU para 100 passos tempora

	Kernel (ms)			Cópias (ms)		
Malha	**Original**	**Persistente**	**Otimizada**	**Original**	**Persistente**	**Otimizada**
256 × 256	0,778	0,791	0,314	9,071	0,089	0,089
512 × 512	2,276	2,274	0,558	39,774	0,385	0,396
1024 × 1024	7,137	7,094	2,35	232,412	2,184	2,207
2048 × 2048	27,553	27,412	8,71	905,717	9,703	9,264
4096 × 4096	110,1	110,64	34,177	3555,353	34,129	50,457

**Medido pelo NVIDIA Nsight Systems. Os valores foram obtidos em uma execução separada e não incluem todos os custos contabilizados no tempo de solve.*

Figura 5: Mediana de três execuções, com 100 passos temporais com tempos em segundos

Malha	Original com cópias (s)	Persistente (s)	Otimizada (s)	Aceleração
256 × 256	0,306995	0,288012	0,273794	1,12×
512 × 512	0,34901	0,2878	0,288159	1,21×
1024 × 1024	0,546764	0,292738	0,300905	1,82×
2048 × 2048	1,218591	0,313486	0,305858	3,98×
4096 × 4096	3,899758	0,42428	0,35626	10,95×

Os resultados mostram que manter os dados na GPU trouxe o maior ganho, e reorganizar os laços para acesso contíguo acelerou ainda mais os kernels. Na malha 4096 × 4096, a versão otimizada foi 10,95 vezes mais rápida que a original, com o mesmo erro L2 registrado.

O gargalo da versão original está nas transferências. Para 4096 × 4096, o perfil registrou 100 cópias da CPU para GPU e 100 da GPU para CPU, somando 3555,353 ms. A versão persistente fez apenas uma transferência em cada direção, somando 34,129 ms, uma redução de aproximadamente 99% no tempo de cópias.

O acesso contíguo melhora o processamento na GPU. Na mesma malha, o tempo acumulado dos 100 kernels caiu de 110,640 ms, na persistente, para 34,177 ms, na otimizada. Isso representa kernels 3,24 vezes mais rápidos e uma redução adicional de 16% no tempo de solve.

Nas malhas 512 e 1024, a otimizada teve uma mediana de solve ligeiramente maior que a persistente, apesar de kernels mais rápidos. O tempo de solve inclui inicialização do dispositivo, runtime e sincronizações. Esses custos têm bastante peso nas malhas pequenas. Com apenas três repetições, essas diferenças pequenas não sustentam uma conclusão firme sobre superioridade entre as duas versões.

Apesar de não ter sida incluída na figura, a precisão foi preservada nos valores registrados onde todas as versões e repetições apresentaram o mesmo erro L2 para cada malha. O erro variou de $1{,}578313 \times 10^{-9}$, em 256, até $7{,}816010 \times 10^{-11}$, em 4096.

Olhando para a segunda tabela, podemos perceber que a versão otimizada também melhora o acesso à memória, reduzindo o tempo dos kernels em cerca de 69% em relação à persistente nessa malha. Esses resultados mostram que as duas otimizações são complementares: uma reduz a movimentação de dados e a outra acelera o processamento na GPU.

Figura 6: heatCommon.c

```c
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <omp.h>
#include "heatCommon.h"
#define PI acos(-1.0)
#define LINE "-------------------\n"

// Configura o problema, executa os passos e verifica a solução na CPU.
int main(int argc, char *argv[]) {

 // Tempo total: inclui configuração, inicialização, solve e norma do erro.
 // omp_get_wtime retorna o tempo de parede em segundos.
 double start = omp_get_wtime();

 // Valores padrão: n células internas em cada direção e nsteps passos.
 int n = 1000;

 // O intervalo físico total permanece 0,5 ao variar o número de passos.
 int nsteps = 10;
```

```c
// Lê os mesmos parâmetros usados pelo script: lado da malha e passos.
if (argc != 1 && argc != 3) {
  fprintf(stderr, "Uso: %s [n passos]\n", argv[0]);
  return EXIT_FAILURE;
}
if (argc == 3) {
  char *fim_n, *fim_passos;
  long valor_n = strtol(argv[1], &fim_n, 10);
  long valor_passos = strtol(argv[2], &fim_passos, 10);
  if (*fim_n || *fim_passos || valor_n < 1 || valor_n > 4096 ||
      valor_passos < 1 || valor_passos > 1000000) {
    fprintf(stderr, "Informe 1 <= n <= 4096 e 1 <= passos <= 1000000.\n");
    return EXIT_FAILURE;
  }
  n = (int)valor_n;
  nsteps = (int)valor_passos;
}

//
// Parâmetros físicos e discretização.
//
double alpha = 0.1;         // Coeficiente de difusão térmica.
double length = 1000.0;     // Lado do domínio quadrado.
double dx = length / (n+1);  // Espaçamento entre pontos internos e bordas.
double dt = 0.5 / nsteps;    // Passo temporal: tempo físico total de 0,5.


// r é o coeficiente adimensional da discretização explícita.
// Para este stencil 2D com espaçamentos iguais, a estabilidade exige r <= 1/4.
double r = alpha * dt / (dx * dx);

// Registra os parâmetros utilizados para permitir interpretar os resultados.
printf("\n");
printf(" MMS heat equation\n\n");
printf(LINE);
printf("Problem input\n\n");
printf(" Grid size: %d x %d\n", n, n);
printf(" Cell width: %E\n", dx);
printf(" Grid length: %lf x %lf\n", length, length);
printf("\n");
printf(" Alpha: %E\n", alpha);
```

```
printf("\n");
printf(" Steps: %d\n", nsteps);
printf(" Total time: %E\n", dt*(double)nsteps);
printf(" Time step: %E\n", dt);
printf(LINE);

// Verifica o limite do esquema explícito bidimensional.
printf("Stability\n\n");
printf(" r value: %lf\n", r);
if (r > 0.25)
  printf(" Warning: unstable\n");
printf(LINE);



// Aloca os dois campos na memória da CPU. A posição (i,j) fica em i+j*n.
// u é a entrada do passo; u_tmp recebe a saída, sem sobrescrever a entrada.
const size_t cells = (size_t)n * n;
double *u     = malloc(sizeof(double)*cells);
double *u_tmp = malloc(sizeof(double)*cells);
if (!u || !u_tmp) {
  fprintf(stderr, "Falha ao alocar os campos.\n");
  free(u);
  free(u_tmp);
  return EXIT_FAILURE;
}

// Inicializa o campo senoidal e zera o segundo array na CPU.
initial_value(n, dx, length, u);
zero(n, u_tmp);

//
// Evolução temporal pelo esquema explícito.
//

// Mede somente o laço temporal: inclui cópias, kernels, sincronização
// e custos do runtime, inclusive os da primeira chamada ao dispositivo.
double tic = omp_get_wtime();
evolve(n, nsteps, alpha, dx, dt, &u, &u_tmp);
// A verificação numérica abaixo fica fora do tempo de solve.
double toc = omp_get_wtime();
```

```
  //
  // Compara o campo final com a solução analítica da MMS.
  //
  double norm = l2norm(n, u, nsteps, dt, alpha, dx, length);

  // Encerra a medição total após a verificação da solução.
  double stop = omp_get_wtime();

  // Imprime o erro e os dois tempos, extraídos pelo script do experimento.
  printf("Results\n\n");
  printf("Error (L2norm): %E\n", norm);
  printf("Solve time (s): %lf\n", toc-tic);
  printf("Total time (s): %lf\n", stop-start);
  printf(LINE);

  // Libera ambas as alocações, independentemente das trocas de ponteiros.
  free(u);
  free(u_tmp);

}


// Condição inicial senoidal da solução manufaturada (MMS).
// Os pontos internos começam em dx; as bordas externas têm temperatura zero.
void initial_value(const int n, const double dx, const double length, double * restrict u) {

  double y = dx;
  for (int j = 0; j < n; ++j) {
    double x = dx; // Coordenada física da primeira célula interna da linha.
    for (int i = 0; i < n; ++i) {
      u[i+j*n] = sin(PI * x / length) * sin(PI * y / length);
      x += dx;
    }
    y += dx; // Avança para a próxima linha da malha.
  }

}


// Preenche o array com zeros na CPU antes do laço temporal.
void zero(const int n, double * restrict u) {
```

```
  for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
      u[i+j*n] = 0.0;
    }
  }

}


// Solução analítica: o perfil senoidal decai exponencialmente com o tempo.
// Serve como referência para verificar o campo calculado numericamente.
double solution(const double t, const double x, const double y, const double alpha, const
double length) {

  return exp(-2.0*alpha*PI*PI*t/(length*length)) * sin(PI*x/length) * sin(PI*y/length);

}


// Calcula na CPU a norma euclidiana do erro: sqrt(soma((u - solução)^2)).
// Não normaliza pelo número de células nem pondera pela área de cada célula.
double l2norm(const int n, const double * restrict u, const int nsteps, const double dt, const
double alpha, const double dx, const double length) {

  // Instante físico final, igual a 0,5 para os parâmetros deste programa.
  double time = dt * (double)nsteps;

  // Acumula os quadrados das diferenças em todas as células internas.
  double l2norm = 0.0;

  // Percorre o campo e avalia a referência nas mesmas coordenadas físicas.
  double y = dx;
  for (int j = 0; j < n; ++j) {
    double x = dx;
    for (int i = 0; i < n; ++i) {
      double answer = solution(time, x, y, alpha, length);
      l2norm += (u[i+j*n] - answer) * (u[i+j*n] - answer);

      x += dx;
    }
```

```
    y += dx;
  }

  return sqrt(l2norm);

}
```

Figura 7: heatCommon.h

```
// Interface comum às três versões da tarefa 20.
// serve para evitar múltiplas inclusões do mesmo arquivo de cabeçalho.
#pragma once

void initial_value(int n, double dx, double length, double * restrict u);
void zero(int n, double * restrict u);
double solution(double t, double x, double y, double alpha, double length);
double l2norm(int n, const double * restrict u, int nsteps, double dt,
              double alpha, double dx, double length);

// Cada versão implementa a evolução e devolve os dois ponteiros atualizados.
void evolve(int n, int nsteps, double alpha, double dx, double dt,
            double **campo, double **auxiliar);
```